CS 24302 : COMPILER DESIGN LABORATORY

Lab Session 5 : Theme : Writing Grammars for various constructs of C Language

Scheduled : August 27 and 28, 2026 (in case of a holiday, Lab 5 is to be scheduled on an alternate date.)

Faculty Team : Bikas Kanti Sarkar, Indrajit Mukherjee, Prashant Pranav, Mohd. Shah Fahad, Supreeti Kamilya and Supratim Biswas

Research Scholar : Swarna Aishwarya Twinkle

Recap :

1. Learn the basic structure of a yacc script and the type of information that is to be supplied in the different placeholders.
2. Get familiar with the manner in which terminals and nonterminals are specified in yacc file. Understand how tokens are communicated between a scanner and its parser. Write a lex script to supply one token at a time to the parser. Use both the scripts to generate a functional parser.
3. Learn a few basic means of communication between yylex() and yyparse(). Examine the contents of the files, "y.tab.h" and "y.output", that is created by yacc. Experiment with the generated parser and examine its behaviour.

Objectives :

1. Practice writing CFG for more features of C language.
2. Add rules in the yacc script to manually draw the parse tree along with parsing.
3. Experiment with writing lex and yacc scripts for processing C code fragments

Lab Assignment 5 : Write a Context Free Grammars to generate C declarations of scalars, arrays and pointers. Write lex and yacc scripts to generate a scanner and a parser and study the behaviour of the parser experimentally.

Read the handout for learning how to manually draw a parse tree by writing suitable actions in the yacc script.

P1. Use the lex and yacc scripts written by you as solution of P2 of Lab4 to manually create a parse tree from the output of the parser.

P2. Declarations of scalar identifiers., some are initialized while others are not. Consider the following declarations as input :

```
int i = 10, j;
char comma = ',', _semicolon;
float pi, g = 9.81, comma;
```

Identify the tokens and grammar rules to write the lex and yacc scripts. Generate the parser and test its performance on the above input and also on some inputs created by you.

P3. Construct a parser to handle declaration of multidimensional arrays and scalars, where all are uninitialized. Your parser should be able to parse declarations of the following form :

```
int a, X[10], b;
char name[10], strings[20][30], ch;
float g[1], f[5][10][8], y;
```

Identify the tokens and grammar rules to write the lex and yacc scripts. Generate the parser and test its performance on the above input and also on some inputs created by you.

P4. Extend the scripts of **P3** to handle pointers also along with uninitialized declarations of scalars and arrays. For example, your parser should be able to parse declarations such as :

```
int *a, *X[10], b;
char name[10], *strings[20][30], ch;
float *g[1], f[5][10][8], *y;
```

Bonus Problem

The medical test report processor was designed as a lex script in an earlier lab. You are required to write a lex and yacc scripts to generate a parser that is capable of producing outputs for the following inputs. Examine the input-output carefully, make your observations before starting your design.

Input reports	Output of the parser
01 Jan 2026 Anil BS(F) 48 BS(PP) 101 HBA1C 5.5 05 Feb 2026 Mita BS(F) 108 BS(PP) 150 HBA1C 6.4 11 Mar 2026 Sunil BS(F) 119 BS(PP) 170 HBA1C 6.1 29 Apr 2026 Seetha BS(F) 113 BS(PP) 140 HBA1C 6.5 10 Aug 2026 Shyam BS(F) 112 BS(PP) 131 HBA1C 7.1	01 Jan 2026 Anil BS(F) 48 BS(PP) 101 HBA1C 5.5 Data of patient No 1 05 Feb 2026 Mita BS(F) 108 BS(PP) 150 HBA1C 6.4 Data of patient No 2 11 Mar 2026 Sunil BS(F) 119 BS(PP) 170 HBA1C 6.1 Data of patient No 3 29 Apr 2026 Seetha BS(F) 113 BS(PP) 140 HBA1C 6.5 Data of patient No 4 10 Aug 2026 Shyam BS(F) 112 BS(PP) 131 HBA1C 7.1 Data of patient No 5
BS(F) 48 01 Jan 2026 Anil BS(PP) 101 HBA1C 5.5 05 Feb 2026 BS(PP) 150 BS(F) 108 HBA1C 6.4 Mita Sunil HBA1C 6.1 BS(PP) 170 BS(F) 119 11 Mar 2026 HBA1C 6.5 BS(F) 113 BS(PP) 140 29 Apr 2026 Seetha BS(PP) 131 HBA1C 7.1 10 Aug 2026 Shyam BS(F) 112	BS(F) 48 01 Jan 2026 Anil BS(PP) 101 HBA1C 5.5 Data of patient No 1 05 Feb 2026 BS(PP) 150 BS(F) 108 HBA1C 6.4 Mita Data of patient No 2 Sunil HBA1C 6.1 BS(PP) 170 BS(F) 119 11 Mar 2026 Data of Patient No 3 HBA1C 6.5 BS(F) 113 BS(PP) 140 29 Apr 2026 Seetha Data of patient No 4 BS(PP) 131 HBA1C 7.1 10 Aug 2026 Shyam BS(F) 112 Data of patient No 5
BS(F) 48 01 Jan 2026 Anil BS(PP) 101 HBA1C 5.5 05 Feb 2026 BS(PP) 150 BS(F) 108 HBA1C 6.4 Mita Sunil HBA1C 6.1 11 Mar 2026 BS(F) 113 BS(PP) 140 29 Apr 2026 Seetha HBA1C 7.1 10 Aug 2026 Shyam BS(F) 112	BS(F) 48 01 Jan 2026 Anil BS(PP) 101 HBA1C 5.5 Data of patient No 1 05 Feb 2026 BS(PP) 150 BS(F) 108 HBA1C 6.4 Mita Data of patient No 2 Sunil HBA1C 6.1 11 Mar 2026 Data of Patient No 3 BS(F) 113 BS(PP) 140 29 Apr 2026 Seetha Data of patient No 4 HBA1C 7.1 10 Aug 2026 Shyam BS(F) 112 Data of patient No 5

End of the Experiment 5